Low-Rank and Tensor Network Methods for Differential Equations

final report

Soeun Jang
Virginia Tech
sophiaj04@vt.edu

June 2026

Contents

1 Lab 1: SVD, Taylor Series, and Low-Rank Truncation **2**
- 1.1 Objectives . . . 2
- 1.2 Background . . . 2
  - 1.2.1 Spatial Discretization . . . 2
  - 1.2.2 Taylor Series Time-Stepping . . . 3
- 1.3 Part 1: LLRSVD Type and Truncation . . . 3
  - 1.3.1 Low-Rank SVD Representation . . . 3
  - 1.3.2 LLRSVD Struct (Conceptual) . . . 3
  - 1.3.3 Frobenius-Norm Truncation . . . 3
- 1.4 Part 2: Taylor Series and Truncation Tolerance . . . 4
  - 1.4.1 Full-Rank Example . . . 4
  - 1.4.2 Singular Values of Taylor Terms . . . 4
- 1.5 Part 3: Adding Low-Rank Matrices (trunc_sum) . . . 4
- 1.6 Part 4: Step-and-Truncate Simulation . . . 4
- 1.7 Part 5: Solid-Body Rotation . . . 5

2 Lab 2: Low-Rank Sensitivities and Gradient Computation **6**
- 2.1 Objectives . . . 6
- 2.2 Background: Parameter-Dependent PDE . . . 6
  - 2.2.1 Cost Functional and Sensitivity Matrix . . . 6
- 2.3 Semi-Discrete System and Sensitivity Equation . . . 7
- 2.4 Dense Reference Solver . . . 7
- 2.5 Step-and-Truncate RK4 for Low-Rank X . . . 7
- 2.6 Gradient from Low-Rank Factors . . . 7
- 2.7 Steepest Descent with Armijo Line Search . . . 8

3 Lab 3: Tucker Tensors **9**
- 3.1 Objectives . . . 9
- 3.2 Tensors and Mode-n Unfolding . . . 9
- 3.3 Mode-n Product . . . 9
- 3.4 Tucker Decomposition and HOSVD . . . 10
  - 3.4.1 HOSVD Algorithm . . . 10
- 3.5 Tucker3 Struct and Reconstruction . . . 10
- 3.6 Tucker Rounding . . . 10
- 3.7 Inner Product and Norm in Tucker Format . . . 11
- 3.8 Sum and Recompression . . . 11
- 3.9 3D Advection in Tucker Format . . . 11

Chapter 1

Lab 1: SVD, Taylor Series, and Low-Rank Truncation

1.1 Objectives

This chapter helps to understand:

- Implement the LLRSVD data structure and use it to represent matrices in a compressed low-rank format.
- Truncate a low-rank representation to a prescribed Frobenius-norm tolerance without recomputing the SVD.
- Add a list of low-rank matrices and recompress the result in a single pass.
- Explain how the Taylor series order p determines the per-term SVD truncation tolerances, and verify this experimentally.
- Build a Step-and-Truncate time-stepper for a 2D PDE that keeps the solution in low-rank form throughout.
- Apply the solver to solid-body rotation and study how the effective rank evolves in time.

1.2 Background

Consider the two-dimensional constant-coefficient advection equation

$$u_t = u_x + u_y, \qquad (x, y) \in [0, 2\pi]^2,\ t > 0, \tag{1.1}$$

with periodic boundary conditions and initial condition $u_0(x, y) = u(x, y, 0)$. The exact solution is the rigid translation

$$u(x, y, t) = u_0(x + t, y + t). \tag{1.2}$$

1.2.1 Spatial Discretization

The domain is discretized on an $m \times n$ uniform mesh with grid points

$$x_i = (i-1)h_x, \quad y_j = (j-1)h_y, \quad h_x = \frac{2\pi}{m}, \quad h_y = \frac{2\pi}{n}.$$

Grid values $W_{ij} \approx u(x_i, y_j, t)$ are stored as an $m \times n$ matrix W. Differentiation matrices $D_x \in \mathbb{R}^{m\times m}$ and $D_y \in \mathbb{R}^{n\times n}$ satisfy

$$\partial_x W \approx D_x W, \qquad \partial_y W \approx W D_y^T.$$

Define the semidiscrete operator

$$\mathcal{L}(W) = D_x W + W D_y^T.$$

1.2.2 Taylor Series Time-Stepping

Advancing from $t = 0$ by one step Δt via a Taylor series of order p:

$$W^{(0)} = W(0), \tag{1.3}$$

$$W^{(k)} = \mathcal{L}\big(W^{(k-1)}\big), \qquad k = 1, \ldots, p, \tag{1.4}$$

$$W(\Delta t) \approx \sum_{k=0}^{p} \frac{(\Delta t)^k}{k!} W^{(k)}. \tag{1.5}$$

1.3 Part 1: LLRSVD Type and Truncation

1.3.1 Low-Rank SVD Representation

A low-rank SVD representation stores a matrix $A \approx U\Sigma V^T$ through three factors:

- $U \in \mathbb{R}^{m \times r}$ with orthonormal columns,
- $\Sigma = \mathrm{diag}(\sigma_1, \ldots, \sigma_r)$ with $\sigma_1 \geq \cdots \geq \sigma_r > 0$,
- $V \in \mathbb{R}^{n \times r}$ with orthonormal columns.

The dense matrix is recovered as $U\mathrm{diag}(S)V^T$, where S is the vector of singular values.

1.3.2 LLRSVD Struct (Conceptual)

In Julia, one defines:

```
mutable struct LLRSVD
    U :: Matrix{Float64}  # m x r, orthonormal columns
    S :: Vector{Float64}  # length r, singular values (descending)
    V :: Matrix{Float64}  # n x r, orthonormal columns
    m :: Int
    n :: Int
    r :: Int
end
```

1.3.3 Frobenius-Norm Truncation

Given singular values $\sigma_1 \geq \cdots \geq \sigma_R$, seek the smallest rank r such that

$$\|A - A_r\|_F^2 = \sum_{i=r+1}^{R} \sigma_i^2 \leq \mathrm{TOL}, \tag{1.6}$$

where A_r is the truncated rank-r approximation.

1.4 Part 2: Taylor Series and Truncation Tolerance

1.4.1 Full-Rank Example

For $m = n = 64$, consider the initial condition

$$u_0(x, y) = \sin(x)\sin(y),$$

which has rank 1 as a matrix: $W(0) = \sin(x)\sin(y)^T$.
Constructing D_x, D_y and verify that

$$\left\| D_x W(0) + W(0) D_y^T - \cos(x)\sin(y)^T - \sin(x)\cos(y)^T \right\|_F$$

is small.

1.4.2 Singular Values of Taylor Terms

For a fixed Δt, compute the Taylor terms

$$T_k = \frac{(\Delta t)^k}{k!} W^{(k)}, \qquad k = 0, \ldots, p.$$

For each k, compute the SVD of T_k and study:

- decay of singular values,
- decay of $\|T_k\|_F$ with k,
- choice of per-term truncation tolerance so that truncation error does not dominate the Taylor truncation error $O((\Delta t)^{p+1})$.

1.5 Part 3: Adding Low-Rank Matrices (trunc_sum)

Implementing a function `trunc_sum(terms, TOL)` that:

- sums a list of LLRSVD matrices,
- recompresses the result to tolerance TOL,
- avoids forming the full $m \times n$ matrix.

1.6 Part 4: Step-and-Truncate Simulation

Building a full Step-and-Truncate solver for the 2D transport equation, comparing:

- Strategy A: truncate each Taylor term first, then sum;
- Strategy B: sum all terms, then truncate once.

1.7 Part 5: Solid-Body Rotation

For the PDE

$$u_t = yu_x - xu_y, \qquad (x, y) \in [-6, 6]^2,$$

with velocity field $v = (y, -x)$, the exact solution is

$$u(x, y, t) = u_0(x \cos t + y \sin t,\ -x \sin t + y \cos t).$$

Extending the low-rank machinery to this variable-coefficient case and studying rank evolution and error over one full revolution.

Chapter 2

Lab 2: Low-Rank Sensitivities and Gradient Computation

2.1 Objectives

This chapter helps to understand:

- Derive the matrix sensitivity equation for a parameter-dependent hyperbolic PDE.
- Implement a dense reference solver for the joint (u, X) system.
- Diagnose compressibility of $X(t)$ via its singular spectrum and numerical rank.
- Build a fixed-rank step-and-truncate RK4 integrator for u and $X \approx USV^T$.
- Evaluate the gradient $\nabla_\alpha J$ directly from low-rank factors at $O((n+p)r)$ cost.
- Use the cheap gradient in a steepest-descent loop with Armijo line search.

2.2 Background: Parameter-Dependent PDE

Consider

$$u_t + A(x)u_x = 0, \qquad x \in [-5, 5) \text{ (periodic)}, \quad 0 < t < T, \quad u(x, 0) = u_0(x), \tag{2.1}$$

where $A(x)$ is the wave speed. After discretization on n points, the state $u(t) \in \mathbb{R}^n$ and parameters

$$\alpha = [A(x_1), \ldots, A(x_p)] \in \mathbb{R}^{1\times p}, \quad p = n.$$

2.2.1 Cost Functional and Sensitivity Matrix

Define

$$J(u(T); u_0) = \frac{1}{2}\|u(T) - u_0\|_2^2.$$

By the chain rule,

$$\frac{\partial J}{\partial \alpha} = (u(T) - u_0)^T \frac{\partial u(T)}{\partial \alpha} = (u(T) - u_0)^T X(T),$$

where $X(t) := \partial u(t)/\partial \alpha \in \mathbb{R}^{n\times p}$.

2.3 Semi-Discrete System and Sensitivity Equation

For the variable-coefficient advection example,

$$\frac{du}{dt} = -\mathrm{diag}(a)\,(Du), \qquad u(0) = u_0,$$

with $a_j = a(x_j)$. The sensitivity matrix X satisfies

$$\frac{dX}{dt} = -\mathrm{diag}(Du) - \mathrm{diag}(a)\,DX, \qquad X(0) = 0.$$

2.4 Dense Reference Solver

Implementing a dense RK4 solver for the coupled (u, X) system:

- CFL-based time step $\Delta t = h/\max_j |a_j|$,
- total time $T = 10$,
- track singular values $\sigma_k(X(t))$ and numerical rank

$$r_\varepsilon(t) = \min\{k : \sigma_{k+1}(X(t)) \le \varepsilon\,\sigma_1(X(t))\}$$

for various ε.

2.5 Step-and-Truncate RK4 for Low-Rank X

Represent $X \approx USV^T$ with $U, V \in \mathbb{R}^{N\times r}$, $S \in \mathbb{R}^{r\times r}$, and propagate:

- advance u with RK4,
- at each stage, update low-rank factors for X using:

$$\frac{dX}{dt} = -\mathrm{diag}(Du) - \mathrm{diag}(a)\,DX,$$

- concatenate factors, perform thin QR and small SVD,
- truncate back to fixed rank r.

Compare:

- wall-clock time per step vs. dense solver,
- relative error $\|X_r(T) - X_{\mathrm{dense}}(T)\|_F / \|X_{\mathrm{dense}}(T)\|_F$,
- error growth vs. time for different r.

2.6 Gradient from Low-Rank Factors

If $X(T) \approx USV^T$, then

$$\nabla_\alpha J(T) = X(T)^T(u(T) - u_0) = V\left(S^T\left(U^T(u(T) - u_0)\right)\right),$$

which costs $O((N+p)r)$ flops and never forms $X(T)$ explicitly.

2.7 Steepest Descent with Armijo Line Search

At iteration k, with parameters α_k:

- search direction: $d_k = -\nabla_\alpha J(\alpha_k)$,
- step length η_k chosen by backtracking:

$$J(\alpha_k + \eta_k d_k) \leq J(\alpha_k) + c_1 \eta_k \nabla_\alpha J(\alpha_k)^T d_k, \quad c_1 \in (0,1),$$

- start from η_0 (e.g. 1) and reduce by factor $p \in (0,1)$ until Armijo condition holds.

Chapter 3

Lab 3: Tucker Tensors

3.1 Objectives

This chapter helps to understand:

- Compute the mode-n unfolding (matricization) of a 3-way tensor.
- Apply the mode-n product to contract a tensor with a matrix along a given mode.
- Implement the Higher-Order SVD (HOSVD) to obtain a Tucker decomposition.
- Truncate a Tucker decomposition to a prescribed multilinear rank and quantify the approximation error.
- Represent a 3D function on a fine grid in compressed Tucker format.
- Perform addition, inner product, and reconstruction directly in Tucker format.

3.2 Tensors and Mode-n Unfolding

A 3-way tensor $A \in \mathbb{R}^{n_1 \times n_2 \times n_3}$ has entries $a_{i_1 i_2 i_3}$. The mode-n unfolding $A^{(n)}$ arranges mode-n fibers as columns:

$$A^{(1)} \in \mathbb{R}^{n_1 \times (n_2 n_3)}, \quad A^{(2)} \in \mathbb{R}^{n_2 \times (n_1 n_3)}, \quad A^{(3)} \in \mathbb{R}^{n_3 \times (n_1 n_2)}.$$

3.3 Mode-n Product

Given $A \in \mathbb{R}^{n_1 \times n_2 \times n_3}$ and $M \in \mathbb{R}^{s \times n_n}$, the mode-$n$ product $B = A \times_n M$ has entries

$$b_{i_1 \ldots i_{n-1} j i_{n+1} \ldots i_3} = \sum_{i_n=1}^{n_n} a_{i_1 \ldots i_n \ldots i_3} M_{j, i_n},$$

and satisfies

$$B^{(n)} = M A^{(n)}.$$

Mode products along different modes commute:

$$(A \times_m M) \times_n N = (A \times_n N) \times_m M, \quad m \neq n.$$

3.4 Tucker Decomposition and HOSVD

A Tucker decomposition of A with multilinear rank (r_1, r_2, r_3) is

$$A \approx G \times_1 U^{(1)} \times_2 U^{(2)} \times_3 U^{(3)},$$

where:

- $G \in \mathbb{R}^{r_1 \times r_2 \times r_3}$ is the core tensor,
- $U^{(n)} \in \mathbb{R}^{n_n \times r_n}$ have orthonormal columns.

3.4.1 HOSVD Algorithm

1. For $n = 1, 2, 3$: compute thin SVD of $A^{(n)} = \widehat{U}^{(n)} \widehat{\Sigma}^{(n)} \widehat{V}^{(n)T}$. Retain leading r_n left singular vectors as $U^{(n)}$.
2. Compute core:

$$G = A \times_1 U^{(1)T} \times_2 U^{(2)T} \times_3 U^{(3)T}.$$

The HOSVD error satisfies

$$\left\| A - G \times_1 U^{(1)} \times_2 U^{(2)} \times_3 U^{(3)} \right\|_F^2 \leq \sum_{n=1}^{3} \sum_{j > r_n} \sigma_j\big(A^{(n)}\big)^2,$$

which is at most 3 times the best rank-(r_1, r_2, r_3) approximation error.

3.5 Tucker3 Struct and Reconstruction

Conceptually, in Julia:

```
mutable struct Tucker3
    G  :: Array{Float64,3}  # r1 x r2 x r3 core
    U1 :: Matrix{Float64}   # n1 x r1
    U2 :: Matrix{Float64}   # n2 x r2
    U3 :: Matrix{Float64}   # n3 x r3
end
```

Reconstruction:

$$A \approx G \times_1 U_1 \times_2 U_2 \times_3 U_3.$$

3.6 Tucker Rounding

Given an existing Tucker decomposition (G, U_1, U_2, U_3), Tucker rounding recompresses the core:

- apply HOSVD to G,
- obtain new core $\widetilde{G}$ and mode matrices $\widehat{U}^{(n)}$,
- set $U_n^{\text{new}} = U_n \widehat{U}^{(n)}$.

This avoids expanding to the full $n_1 \times n_2 \times n_3$ tensor.

3.7 Inner Product and Norm in Tucker Format

For two Tucker tensors

$$A = G \times_1 U^{(1)} \times_2 U^{(2)} \times_3 U^{(3)}, \quad B = H \times_1 W^{(1)} \times_2 W^{(2)} \times_3 W^{(3)},$$

the Frobenius inner product can be written as

$$\langle A, B \rangle_F = \sum_{j_1,j_2,j_3} \sum_{k_1,k_2,k_3} g_{j_1 j_2 j_3} h_{k_1 k_2 k_3} (U^{(1)T} W^{(1)})_{j_1 k_1} (U^{(2)T} W^{(2)})_{j_2 k_2} (U^{(3)T} W^{(3)})_{j_3 k_3},$$

which costs $O(r^6 + nr^2)$ instead of $O(n^3)$. The norm is $\|A\|_F = \sqrt{\langle A, A \rangle_F}$.

3.8 Sum and Recompression

The sum $A + B$ of two Tucker tensors with ranks (r_1^A, r_2^A, r_3^A) and (r_1^B, r_2^B, r_3^B) has a Tucker representation with ranks

$$(r_1^A + r_1^B,\ r_2^A + r_2^B,\ r_3^A + r_3^B),$$

and factor matrices formed by column concatenation $[U_A^{(n)} \mid U_B^{(n)}]$. The combined core is block-diagonal in a suitable sense and can be recompressed via Tucker rounding.

3.9 3D Advection in Tucker Format

For

$$u_t = u_x + u_y + u_z, \qquad (x, y, z) \in [0, 2\pi]^3, \quad u_0(x, y, z) = \sin x \, \sin y \, \sin z,$$

the exact solution is

$$u(x, y, z, t) = u_0(x - t, y - t, z - t).$$

The semidiscrete operator is

$$\mathcal{L}(W) = D_x \times_1 W + D_y \times_2 W + D_z \times_3 W.$$

If W is Tucker with rank (r_1, r_2, r_3), each term in $\mathcal{L}(W)$ is Tucker with rank at most (r_1, r_2, r_3). Implementing:

- `applyL_tucker(W, Dx, Dy, Dz, tol)` using Tucker arithmetic and `tucker_sum`,
- a Step-and-Truncate solver with Taylor order p and tolerance $(\Delta t)^{p+1}$,
- rank-vs-time tracking and error convergence for different Δt.